Emotion Analysis in Text Using Lexicon-Based, Weakly Supervised, and Hybrid Methods

Matei
NLP Final Project

Abstract

This technical report presents an end-to-end implementation for emotion analysis in text. The task is to assign scores for eight basic emotions—anger, anticipation, disgust, fear, joy, sadness, surprise, and trust—and to predict the dominant emotion for each input text. The implementation compares three approaches: a classic weighted lexicon baseline based on the NRC Hashtag Emotion Lexicon, a weakly supervised Multinomial Naive Bayes model trained from lexical emotion associations, and a hybrid score-level combination of the two. The system includes command-line prediction for individual texts and CSV files, reproducible experiments, generated metrics, confusion matrices, and visual artifacts. On a small manually constructed evaluation set of 24 texts from social, news, and blog domains, the lexicon baseline obtains the strongest result, with 0.9167 accuracy and 0.9187 macro-F1. The hybrid method improves over Naive Bayes but does not surpass the lexicon baseline on this dataset. The project demonstrates a complete local implementation that satisfies the requirement of combining classic and learned methods while remaining transparent, reproducible, and easy to inspect.

1 Introduction

Emotion analysis is the task of identifying affective information expressed in text. Unlike binary or ternary sentiment analysis, which usually asks whether a text is positive, negative, or neutral, emotion analysis attempts to capture more specific categories such as fear, joy, sadness, anger, trust, or surprise. This finer-grained representation is useful in applications such as monitoring social-media reactions, analyzing news and blogs, tracking public response to events, or using emotion scores as features for downstream tasks such as hate speech detection, optimism/pessimism analysis, and mental-health-related text analysis.

The selected project topic is *Emotion analysis – detect the emotions in a text*. The implementation focuses on extracting emotion scores for individual emotions, following the eight basic emotion categories used by the NRC emotion lexicon family and related to Plutchik-style emotion categories: anger, anticipation, disgust, fear, joy, sadness, surprise, and trust. The input may be a single text or a CSV file containing many texts. The output contains a normalized score for each emotion, the dominant emotion, a confidence score, and, for the lexicon-based method, interpretable evidence terms.

The project requirement asks for an implementation-oriented project in which the paper describes the problem, the chosen methodology, and technical and experimental details, while the code provides an end-to-end application using both classic and more novel approaches. In response, the implemented system contains:

- a classic lexicon-based emotion scorer using the NRC Hashtag Emotion Lexicon;
- a learned Multinomial Naive Bayes model trained weakly from lexicon entries;
- a hybrid model that combines lexical and learned score distributions;

- a command-line interface for prediction, batch processing, and experiments;
- reproducible evaluation scripts, result tables, confusion matrices, and simple visualizations.

The main contribution is not a claim of state-of-the-art performance, but a complete, reproducible, and interpretable implementation that can be run locally without GPU resources. This is appropriate for the current project scope. A Transformer/GPU extension is discussed as future work, but the submitted system is intentionally lightweight and transparent.

2 Related Work

Emotion analysis is closely related to sentiment analysis, but it models a more detailed affective space. Sentiment analysis traditionally focuses on polarity, such as positive versus negative opinion [5]. Emotion analysis instead asks which emotions are expressed or evoked by text. This distinction is important because two texts can both be negative while expressing different emotions, for example anger versus sadness or fear.

Lexicon-based emotion analysis has a long history in natural language processing. Emotion lexicons represent associations between words or phrases and affective categories. Mohammad and Turney [3] describe the creation of an emotion lexicon using crowdsourcing, and Mohammad and Turney [4] expand this work into the NRC Word–Emotion Association Lexicon. These resources are important because emotion-labeled text datasets can be expensive to create, while lexicons provide reusable affective knowledge.

The NRC Hashtag Emotion Lexicon is especially relevant for social-media-style text. It was automatically generated from tweets containing emotion-word hashtags, and provides weighted associations between terms and eight emotions [1, 2]. This makes it suitable for a project that aims to detect emotions in short texts and social-media-like inputs. The lexicon contains rows of the form `emotion, term, score`, where the score indicates the strength of association between a term and an emotion.

Machine learning approaches provide another way to model emotion. Classic supervised algorithms, such as Naive Bayes, logistic regression, or support vector machines, can be trained on labeled examples. In this project, because a large manually annotated dataset was not used, the Naive Bayes model is weakly supervised: lexicon entries are converted into training examples, and lexical association scores become training weights. This does not replace a fully supervised neural model, but it provides a learned baseline that can be compared with the classic lexicon method.

Recent work in emotion analysis often uses neural models and Transformer architectures, especially when large labeled datasets are available. Such models can capture context better than isolated lexical lookup. However, they require more dependencies, often benefit from GPU acceleration, and are less transparent. The current implementation intentionally prioritizes reproducibility, interpretability, and local execution. A Transformer baseline is therefore treated as future work rather than a dependency of the submitted implementation.

3 Task and Resources

3.1 Task Definition

Given an input text x, the system predicts a score distribution over the set of emotions:

$$E = \{\text{anger}, \text{anticipation}, \text{disgust}, \text{fear}, \text{joy}, \text{sadness}, \text{surprise}, \text{trust}\}.$$

The output is a vector:

$$\mathbf{s}(x) = [s_{\text{anger}}, s_{\text{anticipation}}, \ldots, s_{\text{trust}}],$$

where each score is non-negative and the normalized scores sum to 1 for methods that produce positive evidence. The dominant emotion is:

$$\hat{y} = \arg\max_{e \in E} s_e(x).$$

For practical use, the system returns the full score vector, the dominant emotion, the confidence score $s_{\hat{y}}(x)$, and optional interpretability information.

3.2 NRC Hashtag Emotion Lexicon

The primary resource is the NRC Hashtag Emotion Lexicon v0.2. According to its documentation, it contains word associations with eight emotions: anger, fear, anticipation, trust, surprise, sadness, joy, and disgust. The associations were computed from tweets containing emotion-word hashtags such as `#happy` or `#anger`. Each line has the format:

```
<AffectCategory><tab><term><tab><score>
```

The score indicates the strength of association between the term and the emotion. The implementation loads the lexicon from the local extracted resources directory:

```
../linkuri_extrase/archives/.../NRC-Hashtag-Emotion-Lexicon-v0.2.txt
```

The code also supports a custom lexicon path through the `-lexicon` command-line option or the `EMOTION_LEXICON_PATH` environment variable.

3.3 Evaluation Data

To provide a complete experimental loop, a small manually constructed dataset was added in:

```
rezolvari/data/final_eval_texts.csv
```

It contains 24 short English texts: three examples for each of the eight emotions. The examples are divided into three domains: social, news, and blog. This dataset is intentionally small and transparent. It is suitable for demonstrating the implementation and comparing methods, but it should not be interpreted as a large benchmark.

4 Methodology

The implementation compares three methods: a weighted lexicon baseline, a weakly supervised Naive Bayes classifier, and a hybrid model.

4.1 Text Preprocessing

All methods share a lightweight preprocessing pipeline:

1. URLs are removed so that arbitrary links do not become emotional features.
2. Text is lowercased.
3. Tokens are extracted with a regular expression that preserves hashtags.

4. Repeated characters are normalized, e.g., `happppy` is shortened.

5. Lookup variants are generated, such as both `#happy` and `happy`.

The lexicon method also checks for short-window negation and intensification. If a token is preceded by a negation such as `not`, `never`, or `no`, its contribution is reduced. If it is preceded by an intensifier such as `very`, `really`, or `extremely`, its contribution is increased. These are simple heuristics rather than full syntactic analysis.

4.2 Weighted Lexicon Model

The lexicon method searches each token and its variants in the NRC Hashtag Emotion Lexicon. If token t is associated with emotion e by weight $w(t, e)$, the raw score for e is increased:

$$r_e(x) = \sum_{t \in x} m(t) \cdot w(t, e),$$

where $m(t)$ is a multiplier determined by simple negation or intensifier rules. After all matches are processed, scores are normalized:

$$s_e(x) = \frac{\max(r_e(x), 0)}{\sum_{e' \in E} \max(r_{e'}(x), 0)}.$$

The model also stores evidence terms. This is useful because it allows a user to see which tokens contributed most strongly to the final prediction.

4.3 Weakly Supervised Naive Bayes

The Naive Bayes model is trained from the lexicon itself. Each lexicon entry becomes a weakly supervised example:

$$(\text{term}, \text{emotion}, \text{score}).$$

The term is treated as a short training text, the emotion is treated as the label, and the lexicon score is converted into a training weight. The model uses word features and character n-gram features of length 3, 4, and 5. Character n-grams help the model generalize to related forms, hashtags, and noisy short-text vocabulary.

The model estimates:

$$P(e \mid x) \propto P(e) \prod_{f \in F(x)} P(f \mid e)^{c(f,x)},$$

where $F(x)$ is the set of extracted features and $c(f, x)$ is the count of feature f in text x. Additive smoothing is used to avoid zero probabilities for unseen features. The final log-scores are converted into a probability distribution using softmax.

4.4 Hybrid Model

The hybrid method combines the normalized score distributions from the lexicon model and the Naive Bayes model:

$$\mathbf{s}_{hybrid}(x) = \lambda \cdot \mathbf{s}_{lexicon}(x) + (1 - \lambda) \cdot \mathbf{s}_{NB}(x).$$

The default implementation uses $\lambda = 0.85$, meaning that the hybrid method preserves strong lexical evidence while adding a smaller learned signal. The weight is configurable through:

```
-hybrid-weight
```

This makes it possible to tune the balance between interpretability and learned generalization.

5 Implementation

The project code is contained in the folder `rezolvari`. The implementation uses only Python standard library modules at runtime. The only additional Python package installed during the project was `python-pptx`, used to create the PowerPoint presentation; it is not required by the emotion analysis application.

5.1 Code Structure

File	Role
`emotion_cli.py`	terminal entrypoint
`constants.py`	emotion labels shared across modules
`text.py`	preprocessing and tokenization
`resources.py`	lexicon path resolution and loading
`lexicon_model.py`	weighted lexicon baseline
`nb_model.py`	weakly supervised Naive Bayes
`pipeline.py`	unified analyzer for all methods
`io.py`	CSV reading and prediction writing
`metrics.py`	accuracy, macro-F1, confusion matrix
`experiments.py`	final experiment runner
`visualization.py`	SVG distribution charts
`tests/test_core.py`	unit tests

Table 1: Main implementation files.

5.2 Command-Line Interface

The system can be used from the command line. For a single text:

```
py emotion_cli.py predict -text "I am happy but also afraid." -method hybrid
```

For a CSV file:

```
py emotion_cli.py predict-file
-input data/sample_texts.csv
-output outputs/predictions.csv
-method hybrid
```

For the final experiment:

```
py emotion_cli.py run-experiments
```

The experiment command generates CSV and JSON outputs in `outputs/final_experiment/`, including method comparison, predictions, confusion matrices, per-domain metrics, and SVG charts.

5.3 Testing

The project includes unit tests for:

- the lexicon scoring behavior;
- the Naive Bayes model on toy examples;
- the hybrid pipeline using injected models;
- the metric calculation functions;
- validation of invalid hybrid weights;
- creation of experiment outputs.

The tests are run with:

```
$env:PYTHONPATH='src'; py -m unittest discover -s tests
```

At the time of writing, all 6 tests pass.

6 Experimental Setup

6.1 Compared Methods

The experiment compares three methods:

1. **Lexicon**: the weighted NRC Hashtag Emotion Lexicon scorer.
2. **NB**: the weakly supervised Naive Bayes model trained from lexicon entries.
3. **Hybrid**: score-level combination of lexicon and Naive Bayes scores.

The same preprocessing pipeline and emotion label set are used across all methods.

6.2 Dataset

The evaluation dataset contains 24 labeled texts. It is balanced across the eight emotion labels, with three examples per emotion. It also includes a `domain` column with three values: `social`, `news`, and `blog`. Each domain contains 8 examples.

Property	Value	Description
Rows	24	total examples
Emotion labels	8	anger, anticipation, disgust, fear, joy, sadness, surprise, trust
Examples per emotion	3	balanced label distribution
Domains	3	social, news, blog
Examples per domain	8	balanced domain distribution

Table 2: Evaluation dataset summary.

6.3 Metrics

The main metrics are accuracy and macro-F1. Accuracy measures the fraction of examples where the predicted dominant emotion matches the gold label. Macro-F1 is the average of the F1 scores computed independently for each emotion. Macro-F1 is useful for multi-class emotion analysis because it treats all emotions equally.

The experiment also writes confusion matrices and per-domain metrics to support qualitative analysis.

7 Results

7.1 Overall Comparison

Table 3 shows the main results. The lexicon baseline obtains the best performance on the demo evaluation set, with 0.9167 accuracy and 0.9187 macro-F1. The hybrid model improves over Naive Bayes, but does not surpass the lexicon method. This suggests that the demo texts contain explicit emotion-bearing words that are well covered by the NRC Hashtag Emotion Lexicon.

Method	Accuracy	Macro-F1	Examples
Lexicon	0.9167	0.9187	24
Naive Bayes	0.7917	0.7780	24
Hybrid	0.8750	0.8708	24

Table 3: Overall method comparison on the labeled evaluation set.

7.2 Domain-Level Results

Table 4 shows accuracy by text domain. The lexicon method reaches perfect accuracy on the blog subset and 0.875 on news and social texts. Naive Bayes is weaker on news and social examples. The hybrid model is more stable across domains, obtaining 0.875 accuracy for blog, news, and social examples.

Method	Domain	Accuracy	Macro-F1
Lexicon	blog	1.0000	1.0000
Lexicon	news	0.8750	0.8333
Lexicon	social	0.8750	0.8333
Naive Bayes	blog	0.8750	0.8333
Naive Bayes	news	0.7500	0.6667
Naive Bayes	social	0.7500	0.6667
Hybrid	blog	0.8750	0.8333
Hybrid	news	0.8750	0.8333
Hybrid	social	0.8750	0.8333

Table 4: Results by domain. Each domain contains 8 examples.

7.3 Example Prediction

For the input:

> *I finally received the scholarship and I feel proud happy and hopeful.*

the hybrid method predicts `joy` with confidence 0.5847. The same CSV output includes one column for each emotion score, making the predictions easy to inspect and reuse in reports.

8 Error Analysis and Discussion

The main experimental observation is that the lexicon baseline is strongest on the current dataset. This is expected because many examples were designed to express emotions through explicit words such as *happy*, *furious*, *afraid*, *sad*, *shocked*, or *trust*. Such words are likely to have strong associations in the NRC Hashtag Emotion Lexicon.

The Naive Bayes model performs worse than the lexicon baseline. This is not surprising because it is trained from isolated lexicon terms, not from full manually labeled texts. The model can generalize through character n-grams, but its weak labels are noisy and its training data does not fully represent sentence-level context.

The hybrid model improves over Naive Bayes, showing that the learned signal can be useful when constrained by lexical evidence. However, it does not beat the pure lexicon method on this dataset. This suggests that, for a small evaluation set with explicit emotion words, the interpretable lexicon signal is more reliable than the weakly supervised learned signal. On a larger or noisier dataset, especially with misspellings, hashtags, and less direct vocabulary, the hybrid method may become more competitive.

The confusion matrices generated by the code can be used for more detailed inspection. Likely confusions include anticipation versus fear or surprise, and trust versus fear or sadness, because some words may have associations with multiple emotions depending on context.

9 Limitations

This implementation has several limitations.

First, the main lexicon is English and social-media oriented. It is therefore less suitable for Romanian data or for texts with specialized vocabulary unless additional resources or translation strategies are added.

Second, the evaluation dataset is small. It demonstrates the pipeline and allows reproducible comparison, but it is not a large benchmark. The reported numbers should be interpreted as project-level experimental results, not as state-of-the-art claims.

Third, the Naive Bayes model is weakly supervised from lexicon entries. This means that it learns from word-level associations rather than sentence-level human annotations. As a result, it cannot fully model context, irony, negation scope, or compositional meaning.

Fourth, the lexicon method uses simple heuristics for negation and intensification. These heuristics are useful for an implementation baseline, but they do not replace syntactic or semantic analysis.

Finally, the project does not include a Transformer model. Such a model would likely be stronger on large labeled datasets, but it would require additional dependencies and potentially GPU resources. This is left as future work.

10 Ethical Statement

Automatic emotion analysis should be used carefully. A model that predicts emotional categories from text is not a psychological diagnostic tool. It may be useful for aggregate analysis, exploratory research, or feature extraction, but it should not be used to make high-stakes judgments about individuals.

The implementation uses the NRC Hashtag Emotion Lexicon for educational and research purposes. The lexicon documentation states that the resource may be used freely for non-commercial research and

educational purposes, but it should not be redistributed as a standalone dataset. Therefore, the project code expects the lexicon to be present locally in the extracted resources folder rather than embedding the lexicon directly into the source code.

The system can make errors, especially on ambiguous, sarcastic, multilingual, or context-dependent text. Any downstream use should report uncertainty, inspect examples, and avoid over-interpreting predicted emotions.

11 Future Work

The most natural extension is to add a neural Transformer baseline. For example, a model such as BERT, RoBERTa, or a Twitter-specific emotion model could be fine-tuned on a larger labeled emotion dataset. This would make GPU platforms such as RunPod or Kaggle useful, but they are not necessary for the current implementation.

Another extension is Romanian emotion analysis. This could be approached through a Romanian emotion dataset, translation of lexical resources, multilingual embeddings, or multilingual Transformer models. This would address the low-resource language direction suggested in the project topic.

The evaluation could also be expanded with larger datasets, more domains, and temporal analysis. For example, social-media posts could be grouped by time to track emotion trends, or emotion scores could be used as features in another classification task such as hate speech detection.

Finally, the hybrid model could be tuned more systematically. Instead of using a fixed lexicon weight, one could optimize the interpolation weight on a validation set.

12 Conclusion

This project implements an end-to-end application for emotion analysis in text. It predicts scores for eight emotions and supports both individual and batch prediction. The implementation compares a classic lexicon-based method, a weakly supervised Naive Bayes model, and a hybrid score-combination method.

The results show that the lexicon baseline is the strongest method on the small evaluation dataset, achieving 0.9167 accuracy and 0.9187 macro-F1. The hybrid method improves over Naive Bayes but does not surpass the lexicon baseline. This is a useful result because it shows that simple and interpretable methods can be strong when emotion words are explicit and well covered by the lexicon.

The project satisfies the implementation-focused requirement by providing a runnable application, multiple approaches, reproducible experiments, generated outputs, tests, documentation, and a presentation. Future work can extend the system with larger datasets, Romanian or multilingual resources, and GPU-based Transformer models.

References

[1] Saif M. Mohammad. #emotional tweets. In *Proceedings of the First Joint Conference on Lexical and Computational Semantics*, Montreal, Canada, 2012.

[2] Saif M. Mohammad and Svetlana Kiritchenko. Using hashtags to capture fine emotion categories from tweets. *Computational Intelligence*, 31(2):301–326, 2015.

[3] Saif M. Mohammad and Peter D. Turney. Emotions evoked by common words and phrases: Using mechanical turk to create an emotion lexicon. In *Proceedings of the NAACL HLT 2010 Workshop on*

Computational Approaches to Analysis and Generation of Emotion in Text, pages 26–34, Los Angeles, California, 2010.

[4] Saif M. Mohammad and Peter D. Turney. Crowdsourcing a word–emotion association lexicon. *Computational Intelligence*, 29(3):436–465, 2013.

[5] Bo Pang and Lillian Lee. Opinion mining and sentiment analysis. *Foundations and Trends in Information Retrieval*, 2(1–2):1–135, 2008.

A Reproducibility Commands

The following commands reproduce the main outputs from the implementation folder:

```
cd rezolvari

$env:PYTHONPATH='src'; py -m unittest discover -s tests

py emotion_cli.py run-experiments

py emotion_cli.py predict -text "I trust the team" -method hybrid
```

B Generated Artifacts

The command `run-experiments` creates the following files:

- `metrics.json`: full metrics and confusion matrices;
- `method_comparison.csv`: main comparison table;
- `predictions_lexicon.csv`, `predictions_nb.csv`, `predictions_hybrid.csv`: method-specific predictions;
- `confusion_lexicon.csv`, `confusion_nb.csv`, `confusion_hybrid.csv`: confusion matrices;
- `predicted_distribution_*.svg`: emotion distribution figures.

C PowerPoint Presentation

The slideshow deliverable is stored outside the implementation folder as:

```
Emotion_Analysis_Presentation_Developed_v2.pptx
```

It summarizes the problem, methods, implementation, results, limitations, and future work.